Regime Boundaries for Delayed Utility in Multi-Timescale Memory Routing

Chandana Charitha Peddinti, University of Maryland College Park
Hema Raju Barri, Johns Hopkins University

Abstract

Language-model systems can answer from parametric weights, retrieve external evidence, read and write reusable memory, apply temporary task-time updates, or consolidate stable updates more durably. We study the controller problem: when should one policy choose among these six actions under a reward that combines answer quality, operation cost, forgetting risk, and delayed utility? We evaluate a unified router on frozen official runs covering stable recurrence, public changing-answer evaluation, edit propagation, and broad editing. On official PopQA recurring-memory, the router matches retrieval quality at 0.290 while reducing retrieval calls from 100.0 to 72.2; removing delayed utility drops quality to 0.090. On official FreshQA public main and sequence, retrieval is perfect across all four models, while suppressing delayed utility remains near ceiling at 0.982--0.996 and soft calibration only partially repairs the full router. On official MQuAKE, retrieval again dominates, and on official UniEdit all modes are near ceiling and largely nondiagnostic. These results support a simple empirical law: delayed utility helps when recurrence is stable and support can be reused safely, but retrieval-dominant behavior is more reliable in volatile public-answer and propagation-heavy regimes.

Introduction

Language-model systems can answer from parameters, retrieve external evidence, cache reusable facts, or update themselves temporarily or durably. Existing work usually studies retrieval selection [1,2], editing and memory mechanisms [3,4], or broad surveys of knowledge editing [5] in isolation. We instead study a controller problem: when should one policy choose among parametric answering, retrieval, memory reads, memory writes, temporary adaptation, and guarded durable consolidation under a single objective?

The answer is not a universal-win story. Our frozen results show one clear positive regime and several clear negative ones. On PopQA recurring-memory [6], delayed utility is genuinely useful: the full router matches retrieval quality while cutting retrieval calls substantially, and removing delayed utility collapses performance. On public FreshQA-style changing-answer evaluation derived from FreshLLMs weekly releases [7], retrieval dominates across all four models, and suppressing delayed utility is more reliable than softly calibrating it. On MQuAKE [8], retrieval again dominates, and the gap persists on direct edits, single-hop probes, and multi-hop consequences. On UniEdit [9], nearly every method is already at ceiling, so the benchmark serves mainly as a sanity check rather than a separator.

Those results support a sharper claim than the project originally targeted: delayed utility helps in stable recurring-memory settings, but hurts in public changing-answer and edit-propagation settings; in those regimes, retrieval-dominant behavior wins, and removing delayed utility is more reliable than softly calibrating it.

The public benchmark trust story is also intentionally modest. Our manual FreshQA audit found 24 clean cases, 76 suspect cases, and no direct leakage-risk cases. That rules out a simple leakage explanation for the retrieval-dominant public result, but it does not fully clear the benchmark. Appendix A includes a small deterministic controlled-update diagnostic that is consistent with the same boundary, but the headline evidence in the paper comes from the frozen benchmark surface.

The paper makes four contributions:

- a unified six-action routing formulation over parameters, retrieval, reusable memory, temporary adaptation, and guarded durable consolidation;
- a clean positive result on stable recurring-memory PopQA, where delayed utility preserves quality while reducing repeated retrieval;
- a cross-benchmark boundary result showing that retrieval dominates public changing-answer and propagation-heavy regimes, with soft calibration offering only limited repair relative to outright suppression of delayed utility;
- a frozen evidence package spanning FreshQA public main and sequence runs, MQuAKE, UniEdit, a manual FreshQA audit, and benchmark reliability notes.

Problem Formulation

Each benchmark row is converted to a timestamped episode that matches the implementation schema EpisodeInput. An episode stores the question, gold answer, dataset identifier, timestamp, subject, relation, domain, optional parametric answer and confidence, benchmark hints for recurrence/stability/volatility/contradiction, and a metadata map for regime-specific fields such as confirmation counts or stale-answer markers. When external evidence is available, the episode also carries a list of SupportDoc records; each support document stores a document id, text span, answer string, source name or URL, timestamp, trust score, and retrieval relevance score.

The router chooses one action from:

PARAM_ONLY, READ_MEMORY, RETRIEVE, WRITE_MEMORY, FAST_ADAPT, CONSOLIDATE.

The implemented reward has the form

$$r_t = q_t - c_t - f_t + \beta V_{future}(x_t, a_t),$$

where q_t is exact-match answer quality, c_t is the weighted operation cost, and f_t is the forgetting penalty. In code, the cost term is

$$c_t = 0.4 \ell_t + 0.8 n^{retrieve}_t + 0.7 n^{write}_t + 0.8 n^{adapt}_t + 1.1 n^{consolidate}_t,$$

the forgetting term uses unit weight, and the delayed-utility coefficient is $\beta = 0.9$. The delayed-utility quantity $V_{future}(x_t, a_t)$ is a separate learned estimate of downstream reuse benefit; it is described concretely in the routing policy below.

The design goal is practical rather than purely formal: use one routed interface and one reward function to compare operations that live on different time scales. READ_MEMORY and WRITE_MEMORY access reusable external memory, FAST_ADAPT creates a reversible temporary patch, and CONSOLIDATE promotes a validated temporary patch into the durable bank. Keeping this action inventory fixed across benchmarks is what lets the paper compare stable recurrence, public answer changes, and propagation-heavy edits within one controller.

System And Controller

The system has three explicit knowledge substrates:

- frozen parametric knowledge from the base model,
- persistent external memory,
- a detachable patch bank for temporary and durable updates.

The controller is a LinUCB-style contextual bandit with a separate learned future-value regressor. For each valid action a, it scores

$$score_t(a) = x_t^\top \theta_a + \alpha \sqrt{x_t^\top A_a^{-1} x_t} + prior_a(x_t) + \lambda_e \hat{V}_a(x_t),$$

where x_t is the full RouterFeatures vector, $\alpha = 0.6$, A_a is initialized to the identity matrix, b_a is initialized to zero, and $\theta_a = A_a^{-1} b_a$. The coefficient λ_e is 1.0 for the base router, 0 when delayed utility is disabled, and the public-track calibration scale for router_calibrated. The fixed action prior $prior_a(x_t)$ is action-specific and implementation-defined: it favors confident parametric answers for PARAM_ONLY, aligned/high-hit states for READ_MEMORY, high retrieval quality for RETRIEVE, and stable recurrent regimes for write, adapt, and consolidate actions. After each routed episode, the chosen arm is updated online by $A_a \leftarrow A_a + x_t x_t^\top$ and $b_a \leftarrow b_a + r_t x_t$.

Feature provenance.

The runtime-derived features are the current model confidence, the quality of the top retrieved document, the memory-hit score, the memory-alignment score, the time since the last memory update, and whether the immediately preceding action was a retrieval. The benchmark-provided or metadata-derived features are recurrence, stability, volatility, contradiction risk, source-agreement count, domain-change rate, popularity bin, question-seen status, answer-changed status, weeks since the last change, alias availability, and prior stale-answer availability. The implementation also includes a derived forgetting-risk feature, computed as $\max(0, \text{volatility} + 0.2 \cdot \text{update_expected})$, so that public correction chains and propagation-heavy edits pay a larger mutation penalty than stable recurrence.

Future-value estimator.

Delayed utility is learned by a separate linear regressor for each action. The future-value basis contains five features: recurrence, stability, source-agreement count, volatility, and contradiction risk. Each action starts from a small heuristic weight vector that reflects its intended reuse pattern, and those weights are updated online with learning rate 0.05. The target is the observed future utility over the next three episodes. That utility credits repeated subject/relation reuse and domain updates; on the public track it

gives nonzero credit to write/adapt actions only when the question has been seen before and answer-change or stale-answer evidence is already present, and it requires at least two confirmations before consolidation receives delayed-utility credit.

Public-track calibration.

The calibrated controller does not change the action inventory or the immediate bandit term. It only rescales delayed utility through calibrated_future_value_scale. The scale depends on whether the question was seen before, whether the answer changed since the previous appearance, whether a stale prior answer is available, the update type, confirmation and change counts, recurrence, volatility, contradiction risk, retrieval quality, alias availability, and stability/volatility classes. Stable confirmations can restore the scale up to 0.9, while safe forgetting probes can restore it up to 0.55; unstable public cases can push the scale close to zero.

Execution semantics.

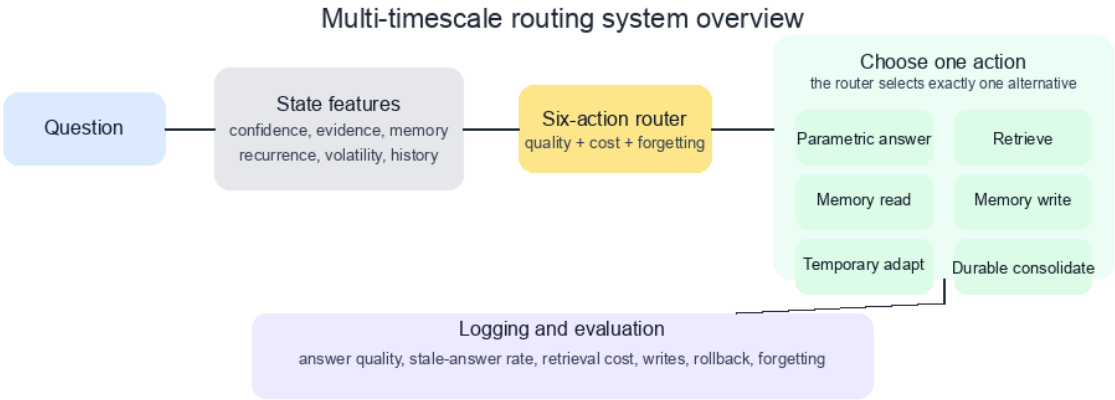
Retrieval uses BM25 over the benchmark-provided support documents or cached local corpora. The controller uses top-k=3 retrieval by default and top-k=5 on freshness/adaptation/consolidation paths; freshness-style tasks also enforce a timestamp filter so only evidence available at the current episode time can be used. On freshness/public benchmarks, the answer is selected directly from the retrieved support documents: retrieve uses the highest trust-plus-relevance document, while adapt uses the latest/high-trust document. On PopQA, MQuAKE, and UniEdit, the retrieved documents are passed to the underlying QA model through an evidence-conditioned prompt. Memory queries return the highest-support, highest-stability, latest record for the current subject--relation pair. PARAM_ONLY first checks active temporary or durable patches, then falls back to the base model. FAST_ADAPT creates a temporary patch whose horizon comes from benchmark metadata. CONSOLIDATE promotes the current temporary patch, creating one on demand if a confirmation episode qualifies. Rollback removes a durable patch when a rollback probe retrieves evidence that disagrees with the promoted answer.

Write and consolidation guards.

Memory writes and consolidations are controlled by deterministic masks rather than by free-form generation. The base write gate requires sufficient retrieval quality, nontrivial recurrence, at least one supporting source, and no existing memory record; the PopQA guardrails then tighten those thresholds further so weak-recurrence cases default back to retrieval. On freshness-style tasks, volatile or corrective updates disable memory writes; on the public track, writes are re-enabled only for answer-changing or stale-answer cases, or for unseen but unusually stable high-confidence cases. Consolidation requires at least two supporting sources, recurrence at least 0.6, stability at least 0.7, volatility at most 0.3, contradiction risk at most 0.3, and no recent rollback. The public track adds stronger conditions: the question must have been seen before, the answer must already have changed, at least one earlier change and two confirmations must be present, and the current episode must be a stable confirmation.

Algorithm 1. Multi-timescale routing policy. The policy scores six actions with one utility objective, applies benchmark-aware masks, executes the chosen operation, and logs the resulting state changes.

1. Build features. Compute the full feature vector x_t from the current episode, world state, retrieval evidence, memory state, change-tracking metadata, and recent action history.
2. Compute regime masks. Apply the benchmark-specific guardrails and mark each action in A as valid or invalid.
3. Score valid actions. For each valid action a, compute $x_t^\top \theta_a + \alpha \sqrt{x_t^\top A_a^{-1} x_t} + prior_a(x_t) + \lambda_e \hat V_a(x_t)$.
4. Select action. Choose $a_t = \arg\max_{a \in A_{valid}} score_t(a)$.
5. Execute action. Run the corresponding parametric, retrieval, memory, or patch operation and collect the answer, evidence ids, touched state, and side effects.
6. Compute reward. Evaluate answer quality, latency and operation cost, forgetting penalty, and delayed utility to obtain r_t.
7. Update estimators. Set $A_{a_t} \leftarrow A_{a_t} + x_t x_t^\top$, $b_{a_t} \leftarrow b_{a_t} + r_t x_t$, recompute θ_{a_t}, and update the action-specific future-value regressor with the observed next-three-episode utility.
8. Log trace. Store the action mask, action scores, feature vector, reward breakdown, evidence ids, state mutations, and benchmark metrics for later aggregation.



System overview. A question and its state features are routed to one of six actions: parametric answering, retrieval, memory read, memory write, temporary adaptation, or durable consolidation. Every episode records action usage, answer quality, stale-answer behavior, rollback, and forgetting-related metrics.

Temporary adaptation.

The implementation keeps FAST_ADAPT lightweight: it attaches a temporary patch with a bounded horizon rather than training a full adapter. This choice keeps the controller claim separate from the heavier engineering trade-offs explored in dedicated editing systems such as MELO and WISE [4,3].

Durable consolidation.

CONSOLIDATE promotes a validated temporary patch into the durable patch bank. This path is rare by design, requires the guard conditions above, and is blocked after rollback.

Rollback.

Correction episodes can invalidate a previously durable answer. The implementation explicitly records whether rollback was triggered, whether a durable patch was used, and whether forgetting probes remain correct afterward.

Benchmarks And Evaluation Protocol

The paper is written from a frozen official evidence surface with fixed datasets, model backbones, prompts, and scoring rules. The benchmark map below summarizes the benchmark set, and the cross-benchmark summary table that follows previews the empirical law that emerges from it.

Benchmark map for the official benchmark surface. The headline evidence comes from frozen runs on PopQA, FreshQA public main and sequence, MQuAKE, and UniEdit.

Benchmark	Regime	Source	Size	Models / seeds	Main question
PopQA	Stable recurrence	Cached public PopQA slice	500	1 / 5	Can delayed utility replace repeated retrieval on recurring facts?
FreshQA main	Volatile public knowledge	Weekly public releases	796	4 / 5	Do changing answers favor retrieval-dominant behavior?
FreshQA sequence	Correction chains	Weekly public releases	796	4 / 5	Does the same boundary survive repeated corrections and forgetting probes?
MQuAKE	Propagation	Princeton-NLP MQuAKE-CF-3k	21000	2 / 5	Do edited facts propagate to single-hop and multi-hop consequences?
UniEdit	Broad editing	qizhou/UniEdit sampled test set	3750	4 / 5	Does the boundary survive a broad-domain editing benchmark?

Cross-benchmark summary from the frozen evidence surface. The key result is not broad router superiority; it is a boundary law showing where delayed utility helps and where retrieval should dominate.

Regime	Benchmark	Best policy	Key evidence	Law component
Stable recurrence	PopQA	Router ties retrieval	0.290 versus 0.290, retrieval 100.0 -> 72.2, and no-V_future falls to 0.090	Delayed utility helps when reuse is stable.
Volatile public knowledge	FreshQA main	Always retrieve	1.000 on all 4 models; calibrated router 0.694--0.794; no-V_future 0.984--0.996	Retrieval should dominate changing-answer public data.
Public correction chains	FreshQA sequence	Always retrieve	1.000 on all 4 models; calibrated router 0.724--0.767; no-V_future 0.982--0.994	Suppressing delayed utility is safer than soft calibration.
Propagation	MQuAKE	Always retrieve	0.551 versus router 0.276--0.284; no-V_future 0.319--0.338	Delayed utility does not transfer safely to propagation-heavy edits.
Broad editing sanity check	UniEdit	Near tie, retrieval best	Quality 0.998--1.000 across all models and modes	The benchmark is nondiagnostic for router superiority.

The benchmark surface has four regimes:

- PopQA recurring-memory: a cached recurring-heavy slice that asks whether reusable support can substitute for repeated retrieval on stable facts;
- FreshQA public main and sequence: frozen public changing-answer evaluation derived from weekly releases, with the sequence slice stressing repeated corrections and forgetting probes;
- MQuAKE: propagation-heavy knowledge editing where the important question is whether an updated fact transfers to single-hop and multi-hop consequences;
- UniEdit: a broad editing benchmark used here as a high-coverage sanity check.

All runs use the same local practical model matrix:

- google/flan-t5-small
- google/flan-t5-base
- Qwen/Qwen2.5-1.5B-Instruct
- HuggingFaceTB/SmolLM2-360M-Instruct

Main tables use five seeds 0,1,2,3,4 and only the allowed baselines:

- always_retrieve
- retrieve_gate on FreshQA only
- router
- router_calibrated
- router_no_v_future

This baseline set is deliberately controlled rather than leaderboard-oriented. always_retrieve and retrieve_gate anchor retrieval-dominant fallback behavior, router and router_calibrated test the same six-action controller with and without public delayed-utility rescaling, and router_no_v_future isolates the delayed-utility term while keeping the rest of the action inventory fixed. Across compared policies, we keep the backbone fixed within each table, expose the same timestamp-constrained evidence, use the same alias and stale-answer rules, and score every mode with the same evaluator. The support layer adds a 100-case manual FreshQA audit together with MQuAKE and UniEdit reliability notes, all cited later in the paper.

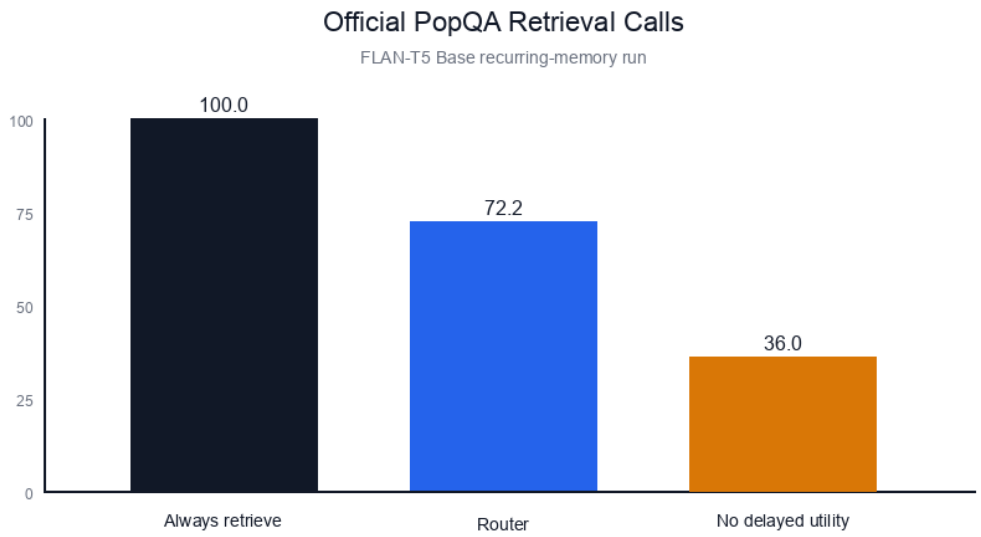
Some 95% confidence intervals collapse to point intervals because all five official seeds produced identical aggregate scores for those settings, so the paired resampling layer had no observed variation to widen the interval.

PopQA: The Positive Stable-Recurrence Regime

PopQA is the clean positive result in the final paper. It isolates a regime where recurrence is stable, reusable support is valuable, and contradiction risk is low. In exactly that setting, delayed utility helps.

Official PopQA recurring-memory result. The router matches retrieval quality while materially reducing retrieval calls through memory reuse.

Model	Policy	Quality	95% CI	Retrieve	Read	Write
FLAN-T5 Base	Retrieval only	0.290	[0.290, 0.290]	100.0	0.0	0.0
FLAN-T5 Base	Retrieve gate	0.290	[0.290, 0.290]	100.0	0.0	0.0
FLAN-T5 Base	Full router	0.290	[0.290, 0.290]	72.2	75.0	49.8
FLAN-T5 Base	Calibrated router	0.290	[0.290, 0.290]	72.2	75.0	49.8
FLAN-T5 Base	Router without delayed utility	0.090	[0.090, 0.090]	36.0	15.0	14.0



Official PopQA retrieval reduction. The full router preserves answer quality while replacing a substantial fraction of repeated retrieval with memory reads and writes.

The official FLAN-T5-Base run gives a precise result. Retrieval-only, the retrieve gate, the full router, and the calibrated router all achieve 0.290 quality. But the router and calibrated router reduce retrieval calls from 100.0 to 72.2 while issuing 75.0 memory reads and 49.8 memory writes. In contrast, the no-V_future ablation (router_no_v_future) falls to 0.090 quality with only 36.0 retrieval calls and much

weaker memory use. The paired sign tests are equally sharp: the full router is tied with retrieval ($p = 1.0$), while suppressing delayed utility is significantly worse ($p = 1.58 \times 10^{-30}$).

This is the strongest direct evidence that delayed utility is not merely decorative. When the environment is stable enough for support to be reused safely, future value changes the preferred policy. The router learns to spend retrieval once, write reusable support, and exploit it later without sacrificing quality. The calibrated router preserves that positive regime because its public-suppression logic does not activate here.

FreshQA Public Main And Sequence: Volatile Public Boundary

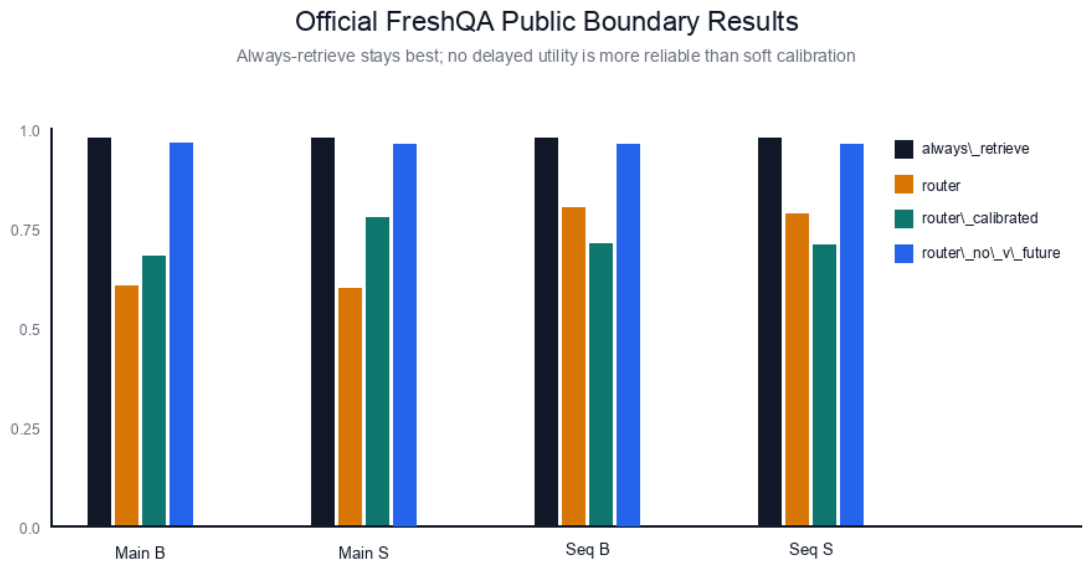
The official FreshQA public runs are the most important boundary evidence in the paper. They test the same controller on frozen public changing-answer data derived from weekly releases, once on a changed-question-heavy main slice and once on a sequence slice with repeated corrections and forgetting probes. Across all four models, the headline result is consistent: retrieval dominates, and suppressing delayed utility is more reliable than softly calibrating it. The accompanying manual audit found 24 clean cases, 76 suspect cases, and no direct leakage-risk cases, so the benchmark is useful and auditable but not fully trust-cleared.

Main slice

Official FreshQA public main results on the primary FLAN matrix. Retrieval is perfect; calibration partially repairs the full router, but the no-V_future ablation remains much closer to the retrieval ceiling.

Model	Policy	Quality	95% CI	Retrieve	Adapt	Stale
FLAN-T5 Base	Retrieval only	1.000	[1.000, 1.000]	796.0	0.0	0.000
FLAN-T5 Base	Retrieve gate	0.132	[0.132, 0.132]	89.0	0.0	0.005
FLAN-T5 Base	Full router	0.616	[0.544, 0.687]	448.6	69.8	0.002
FLAN-T5 Base	Calibrated router	0.694	[0.634, 0.754]	528.4	18.0	0.001
FLAN-T5 Base	Router without delayed utility	0.985	[0.985, 0.985]	740.0	4.0	0.000
FLAN-T5 Small	Retrieval only	1.000	[1.000, 1.000]	796.0	0.0	0.000
FLAN-T5 Small	Retrieve gate	0.134	[0.134, 0.134]	89.0	0.0	0.003
FLAN-T5 Small	Full router	0.612	[0.539, 0.684]	448.6	69.8	0.001
FLAN-T5 Small	Calibrated router	0.794	[0.725, 0.863]	612.4	29.4	0.000
FLAN-T5 Small	Router without delayed utility	0.984	[0.984, 0.984]	739.0	4.0	0.000

On the main slice, always_retrieve is 1.000 on both FLAN backbones and remains 1.000 on Qwen2.5-1.5B and SmolLM2 as well. The full router falls to 0.616 on FLAN-T5-Base and 0.612 on FLAN-T5-Small. Calibration helps relative to the full router, reaching 0.694 and 0.794, but the no-V_future ablation (router_no_v_future) is still far stronger at 0.985 and 0.984. The paired sign tests are decisive: retrieval beats the full router, the calibrated router, and the no-V_future ablation on both FLAN models, while the no-V_future ablation strongly beats both router variants.



Official public-boundary comparison on the primary FLAN matrix. Retrieval is perfect on both FreshQA public slices, while the no-V_future ablation stays much closer to that ceiling than the full or calibrated router.

Sequence slice

Official FreshQA public sequence results on the primary FLAN matrix. On repeated correction chains, suppressing delayed utility is again much safer than the full router, and soft calibration is inconsistent.

Model	Policy	Quality	95% CI	Retrieve	Adapt	Stale
FLAN-T5 Base	Retrieval only	1.000	[1.000, 1.000]	796.0	0.0	0.000
FLAN-T5 Base	Retrieve gate	0.132	[0.132, 0.132]	89.0	0.0	0.005
FLAN-T5 Base	Full router	0.819	[0.708, 0.930]	630.0	84.6	0.000
FLAN-T5 Base	Calibrated router	0.725	[0.647, 0.804]	540.6	26.4	0.000
FLAN-T5 Base	Router without delayed utility	0.984	[0.984, 0.984]	739.0	5.0	0.000
FLAN-T5 Small	Retrieval only	1.000	[1.000, 1.000]	796.0	0.0	0.000
FLAN-T5 Small	Retrieve gate	0.134	[0.134, 0.134]	89.0	0.0	0.003
FLAN-T5 Small	Full router	0.802	[0.687, 0.916]	614.0	82.0	0.000
FLAN-T5 Small	Calibrated router	0.724	[0.643, 0.804]	539.2	26.6	0.000
FLAN-T5 Small	Router without delayed utility	0.982	[0.982, 0.982]	738.0	5.0	0.000

The sequence slice sharpens the same conclusion. Retrieval again stays at 1.000, while the full router reaches 0.819 on FLAN-T5-Base and 0.802 on FLAN-T5-Small. Here calibration is not heroic; it is worse than the full router on both primary FLAN models, falling to 0.725 and 0.724. The no-V_future ablation again stays much closer to retrieval at 0.984 and 0.982, and the corresponding sign tests show it significantly outperforming both router variants. The cross-family appendix extends the same qualitative pattern to Qwen2.5-1.5B and SmolLM2.

Official FreshQA manual audit summary. The audit found no direct leakage-risk cases, but many cases remain suspect, so the public trust story is bounded rather than fully cleared.

Audit field	Count
Always-retrieve correct / router wrong	40
Always-retrieve correct / calibrated-router wrong	25
Alias-heavy or multi-answer	20
Rollback / forgetting / answer-change probes	15
Clean verdicts	24
Suspect verdicts	76
Leakage-risk verdicts	0

The manual audit matters for interpretation. Among 100 audited cases, 24 were labeled clean, 76 were labeled suspect, and none were labeled direct leakage risk. That is enough to reject a simple leakage explanation for the retrieval-dominant result, but not enough to treat the benchmark as fully cleared. The right conclusion is modest: public changing-answer evaluation is hard, retrieval is the safest policy in our current setup, and delayed utility should usually be suppressed rather than softly calibrated in this regime.

MQuAKE: A Retrieval-Dominant Propagation Boundary

MQuAKE asks a different question from FreshQA. The issue is not just whether an answer changed recently; it is whether an updated fact propagates correctly to direct edits, single-hop probes, and multi-hop consequences [8]. In our frozen official runs, this regime is also retrieval-dominant.

Official MQuAKE results. Retrieval remains strongest on overall quality, direct edit success, and multi-hop consequences. Suppressing delayed utility improves on the full router but still does not overtake retrieval.

Model	Policy	Quality	Edit	Single-hop	Multi-hop	Forgetting
FLAN-T5 Base	Retrieval only	0.551	0.214	1.000	0.214	0.00005
FLAN-T5 Base	Full router	0.284	0.188	0.431	0.168	0.00979
FLAN-T5 Base	Calibrated router	0.284	0.188	0.431	0.168	0.00979
FLAN-T5 Base	Router without delayed utility	0.338	0.156	0.643	0.093	0.01086
FLAN-T5 Small	Retrieval only	0.551	0.214	1.000	0.214	0.00005
FLAN-T5 Small	Full router	0.276	0.185	0.433	0.150	0.00226
FLAN-T5 Small	Calibrated router	0.276	0.185	0.433	0.150	0.00226
FLAN-T5 Small	Router without delayed utility	0.319	0.022	0.705	0.031	0.00329

On both FLAN backbones, always_retrieve reaches 0.551 overall quality. The full router reaches only 0.284 on FLAN-T5-Base and 0.276 on FLAN-T5-Small. The calibrated router is identical to the full router here, which means the public-answer calibration rule does not rescue the propagation regime. Suppressing delayed utility improves overall quality to 0.338 on FLAN-T5-Base and 0.319 on FLAN-T5-Small, but those gains still remain far below retrieval.

The benchmark-specific metrics make the boundary clearer. Retrieval reaches edit success 0.214, single-hop accuracy 1.000, and multi-hop accuracy 0.214 on both FLAN backbones. The full router lowers all three. The no-V_future ablation raises single-hop accuracy, but only by sacrificing direct edit success and especially multi-hop accuracy, which falls to 0.093 on FLAN-T5-Base and 0.031 on FLAN-T5-Small. Paired sign tests again show retrieval dominating with effectively zero p-values in both official FLAN bundles.

The main lesson is therefore negative but useful: delayed utility does not transfer safely from stable recurrence to propagation-heavy edits. Even removing delayed utility entirely is only a partial repair, because the strongest safe policy in this regime is still retrieval-dominant.

UniEdit: A Near-Ceiling Broad-Editing Sanity Check

UniEdit broadens the benchmark surface beyond PopQA, FreshQA, and MQuAKE [9]. In our frozen official sample, however, it is almost perfectly solved by every official policy and therefore serves as a sanity check rather than a headline result.

Official UniEdit summary. All policies are near ceiling, so the benchmark is useful for breadth but largely nondiagnostic for router superiority in this setup.

Policy	Quality range	Edit-success range	Generality range	Locality range
Retrieval only	1.0000--1.0000	1.0000--1.0000	1.0000--1.0000	1.0000--1.0000
Full router	0.9990--0.9990	0.9986--0.9992	0.9992--0.9992	1.0000--1.0000
Calibrated router	0.9990--0.9990	0.9986--0.9992	0.9992--0.9992	1.0000--1.0000
Router without delayed utility	0.9980--0.9990	0.9976--0.9992	0.9984--0.9992	0.9984--1.0000

Across FLAN-T5-Small, FLAN-T5-Base, Qwen2.5-1.5B-Instruct, and SmolLM2-360M-Instruct, retrieval-only is exactly 1.000 quality. The router and calibrated router are effectively tied at 0.999, and the no-V_future ablation ranges from 0.998 to 0.999. The benchmark-specific metrics tell the same story: edit success, generality, and locality are all between 0.9976 and 1.0000.

Because the benchmark is so close to saturation, some pairwise differences are statistically detectable but practically tiny. We therefore do not present UniEdit as evidence that the router wins. Instead, we use it as a broad-domain check that the boundary result from FreshQA and MQuAKE is not caused by a trivial implementation failure.

Limitations, Ethics, And Transparency

This paper makes a boundary claim, not a broad-superiority claim. That choice is a limitation, but it is also the clearest honest reading of the frozen evidence.

The router is not broadly best.

Retrieval dominates every official FreshQA run, both official MQuAKE runs, and the official UniEdit runs. The positive claim is therefore narrow: delayed utility helps in stable recurrence, not across all regimes.

Calibration is limited.

On FreshQA public main, calibration partially recovers the full router. On FreshQA public sequence, calibration is inconsistent and is worse than the full router on the primary FLAN matrix. We therefore do not present calibration as a general repair.

The public trust story is bounded.

The manual FreshQA audit found no direct leakage-risk cases, but 76 of 100 audited rows remain suspect. The public benchmark is therefore usable and auditable, but not fully cleared of benchmark-sensitivity concerns.

UniEdit is broad but nondiagnostic here.

The official UniEdit run is near ceiling for all policies. That is useful as a sanity check, but it cannot serve as strong evidence that the router wins or loses meaningfully on broad editing.

Deferred comparisons.

We do not claim results for Qwen2.5-7B-Instruct, faithful MeLLo, or a faithful external editing baseline on UniEdit. Those are important future comparisons, but they are not part of the frozen local evidence surface used here.

Deployment risks.

Persistent memory and durable patches can still create stale caches, incorrect promotions, rollback failures, privacy concerns, and operational opacity. The bounded empirical law in this paper should therefore be read as a diagnostic for when to trust delayed utility, not as a blanket recommendation to deploy more memory mutation.

Reproducibility And Artifact Availability

The paper exposes four evidence surfaces rather than a single undifferentiated bundle: the main evidence surface, an appendix-only supplementary diagnostic, a manual audit bundle, and benchmark-specific reliability notes. Each surface is frozen locally and is named explicitly so that readers can verify the headline claims without rerunning the experiments.

Main evidence surface.

The reportable external evidence surface is the registry-listed set of official bundles under artifacts/tmlr_official/. These bundles cover PopQA recurring memory, FreshQA public main and sequence, MQuAKE, UniEdit, and the cross-benchmark statistics bundle. For any official run root, the first inspection files are manifest.json, aggregate_table.md, run_rows.jsonl, significance.json, and bootstrap.json; MQuAKE and UniEdit additionally expose benchmark-specific aggregates through benchmark_metric_aggregates.json. The cross-benchmark law is summarized in artifacts/tmlr_official/official_statistics_v1/.

Appendix-only supplementary diagnostic.

The only supplementary diagnostic kept outside the main external evidence surface is artifacts/freshness_v1/. It is a deterministic local controlled-update benchmark used as extra arXiv context. It remains useful for understanding stable update behavior, but it is not part of the official external benchmark surface and should not be read as overriding the PopQA, FreshQA public, MQuAKE, or UniEdit results.

Manual audit.

The public benchmark trust story is grounded in artifacts/tmlr_official/freshqa_leakage_audit_manual_v1/. The main inspection files are summary.json, manual_audit_100.md, manual_audit_100.json, and selection_manifest.json. These files expose the bounded trust conclusion directly: no direct leakage-risk cases were found, but many audited rows remain suspect.

Reliability notes.

Benchmark-construction and field-mapping caveats are separated from the main tables. The MQuAKE reliability note lives in artifacts/tmlr_official/mquake_reliability_note_v1/, and the UniEdit snapshot note lives in artifacts/tmlr_official/uniedit_reliability_note_v1/. These notes document how the benchmark inputs were frozen locally and how benchmark-specific metrics should be interpreted.

How to verify.

- Open paper_run_manifest_index.json to see which artifact roots are reportable for the paper.
- Trace each headline claim through paper/shared/claims_to_artifacts.md.
- Inspect the named frozen bundle files directly, starting with manifest.json, aggregate_table.md, run_rows.jsonl, and the significance/bootstrap files for the cited run.
- Use DATASET_MANIFEST.md to verify dataset provenance and frozen input status.

Together these materials make the empirical law auditable from the paper package itself. The manuscript therefore depends on frozen local evidence, not on untracked reruns or external archive links.

Appendix

Cross-family robustness on FreshQA public runs

FreshQA cross-family robustness. The retrieval-dominant boundary survives the extension from the primary FLAN matrix to Qwen2.5-1.5B-Instruct and SmolLM2-360M-Instruct.

Model	Main retrieve	Main calibrated	Main no-V_future	Sequence retrieve	Sequence calibrated	Sequence no-V_future
FLAN-T5 Small	1.000	0.794	0.984	1.000	0.724	0.982
FLAN-T5 Base	1.000	0.694	0.985	1.000	0.725	0.984
Qwen2.5-1.5B	1.000	0.743	0.996	1.000	0.767	0.994
SmolLM2-360M	1.000	0.742	0.985	1.000	0.743	0.987

The qualitative conclusion is unchanged across the four models. Retrieval remains perfect on both public slices. On the main slice, calibration helps relative to the full router, but router_no_v_future remains much closer to retrieval. On the sequence slice, suppression of delayed utility is again safer than the full router, and calibration is inconsistent.

Bundled controlled-update supplementary context

Appendix-only controlled-update diagnostic. This deterministic local benchmark is supplementary context rather than part of the external benchmark surface.

Policy	Quality	Stale	Retrieve	Read	Write	Adapt	Consolidate	Rollback
Parametric only	0.133	0.867	0	0	0	0	0	0
Retrieval only	0.133	0.867	13	0	0	0	0	0
Memory only	0.600	0.400	3	8	3	0	0	0
Fast-adapt only	0.800	0.200	7	0	0	7	0	0
Full router	0.867	0.133	8	6	3	4	2	1

This appendix-only diagnostic is clearly separated from the external benchmark surface. In that deterministic stable-update setting, the full router reaches 0.867 quality versus 0.133 for retrieval-only, 0.600 for memory-only, and 0.800 for fast-adapt-only. The pattern is useful as supplementary context, but it does not override the FreshQA, MQuAKE, or UniEdit evidence.

Evidence taxonomy and inspection guide

The package separates four evidence surfaces explicitly:

- Main evidence surface: registry-listed official bundles under artifacts/tmlr_official/, inspected through manifest.json, aggregate_table.md, run_rows.jsonl, significance.json, and bootstrap.json.
- Appendix-only supplementary diagnostic: artifacts/freshness_v1/.
- Manual audit: artifacts/tmlr_official/freshqa_leakage_audit_manual_v1/.
- Reliability notes: artifacts/tmlr_official/mquake_reliability_note_v1/ and artifacts/tmlr_official/uniedit_reliability_note_v1/.

All headline numbers in the paper are copied from frozen local bundle files only. The mapping from claims to exact source files is recorded in paper/shared/claims_to_artifacts.md, the official run registry is paper_run_manifest_index.json, and dataset provenance is summarized in DATASET_MANIFEST.md. A

reader who wants to verify a claim can therefore follow a short path: registry entry, claims map, frozen bundle files, then dataset manifest.

References

[1] Asai, Akari, Wu, Zeqiu, Wang, Yizhong, Sil, Avirup, Hajishirzi, Hannaneh. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. International Conference on Learning Representations. 2024. https://arxiv.org/abs/2310.11511

[2] Yan, Shi-Qi, Gu, Jia-Chen, Zhu, Yun, Ling, Zhen-Hua. Corrective Retrieval Augmented Generation. CoRR. 2024. https://doi.org/10.48550/arXiv.2401.15884

[3] Wang, Peng, Li, Zexi, Zhang, Ningyu, Xu, Ziwen, Yao, Yunzhi, Jiang, Yong, Xie, Pengjun, Huang, Fei, Chen, Huajun. WISE: Rethinking the Knowledge Memory for Lifelong Model Editing of Large Language Models. CoRR. 2024. https://doi.org/10.48550/arXiv.2405.14768

[4] Yu, Lang, Chen, Qin, Zhou, Jie, He, Liang. MELO: Enhancing Model Editing with Neuron-Indexed Dynamic LoRA. CoRR. 2023. https://doi.org/10.48550/arXiv.2312.11795

[5] Zhang, Ningyu, Yao, Yunzhi, Tian, Bozhong, Wang, Peng, Deng, Shumin, Wang, Mengru, Xi, Zekun, Mao, Shengyu, Zhang, Jintian, Ni, Yuansheng, Cheng, Siyuan, Xu, Ziwen, Xu, Xin, Gu, Jia-Chen, Jiang, Yong, Xie, Pengjun, Huang, Fei, Liang, Lei, Zhang, Zhiqiang, Zhu, Xiaowei, Zhou, Jun, Chen, Huajun. A Comprehensive Study of Knowledge Editing for Large Language Models. CoRR. 2024. https://doi.org/10.48550/arXiv.2401.01286

[6] Mallen, Alex, Asai, Akari, Zhong, Victor, Das, Rajarshi, Hajishirzi, Hannaneh, Khashabi, Daniel. When Not to Trust Language Models: Investigating Effectiveness and Limitations of Parametric and Non-Parametric Memories. CoRR. 2022. https://doi.org/10.48550/arXiv.2212.10511

[7] Vu, Tu, Iyyer, Mohit, Wang, Xuezhi, Constant, Noah, Wei, Jerry W., Wei, Jason, Tar, Chris, Sung, Yun-Hsuan, Zhou, Denny, Le, Quoc V., Luong, Thang. FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation. Findings of the Association for Computational Linguistics: ACL 2024. 2024. https://doi.org/10.18653/v1/2024.findings-acl.813

[8] Zhong, Zexuan, Wu, Zhengxuan, Manning, Christopher D., Potts, Christopher, Chen, Danqi. MQuAKE: Assessing Knowledge Editing in Language Models via Multi-Hop Questions. Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. 2023. https://aclanthology.org/2023.emnlp-main.971/

[9] Chen, Qizhou, Wang, Dakan, Zhang, Taolin, Yan, Zaoming, You, Chengsong, Wang, Chengyu, He, Xiaofeng. UniEdit: A Unified Knowledge Editing Benchmark for Large Language Models. CoRR. 2025. https://doi.org/10.48550/arXiv.2505.12345